

Interview Prep Platform — Frontend (UI) Specification

Version 1.0 · For UI development · Single-user, remote-capable web app

Companion to the Backend Development Specification v2.0. This document defines the screens, components, client-side data models (view models), and exactly which backend endpoints each screen calls. The existing Claude Design output (five screens) is the visual starting point; this extends it to the full product. We (UI) build against the frozen API contract while the backend is developed in parallel.

1. Stack & conventions

Concern	Choice
Framework	React 18 + TypeScript
Routing	React Router
Data/state	TanStack Query (server state) + light local state; no heavy global store
Styling	CSS variables design system (below); Tabler icons
Notes editor	Markdown editor (Toast UI or Milkdown) — stores <code>content_md</code>
Code viewer	Monaco (<code>@monaco-editor/react</code>) — view/edit only, no execution
Auth	JWT in memory + refresh; attach <code>Authorization: Bearer</code> to all calls
API base	<code>/api</code> (same-origin or configured host)

Principles: server state via query hooks (cache, loading, error built in); every screen handles loading / empty / error / **budget-warning** states; optimistic updates for toggles (mark done, exercise done). No `localStorage` for real data — it all persists via the API.

2. Design system (from Claude Design)

```
--paper #F4F2EC    --panel #FFFFFF    --ink #1A1E29    --ink-2 #4A5163    --ink-3 #878D9D
--accent #3D54C9 (indigo)    --green #2B8A5B (done/on-track)    --amber #BA7517 (flagged/review)
--line #E7E3D9
fonts: Space Grotesk (headings) · Inter (body) · JetBrains Mono (labels, codes, IDs)
radius 10px · flat, minimal borders · sentence case · no gradients/heavy shadows
```

Color carries meaning: green = done/ready, amber = needs review/flagged, indigo = primary action, accent tags per topic source (resume / standard / interest / dsa).

3. Client data models (view models)

TypeScript shapes mirroring API responses. These are the contract the UI codes against.

```

type Tag = 'new' | 'refresh' | 'dsa' | 'exp';
type Source = 'resume' | 'standard' | 'interest' | 'custom';

interface Phase { code: string; name: string; icon: string; blurb: string;
  weeks: Week[]; progress: { done: number; total: number }; }
interface Week { code: string; title: string; topics: TopicLite[]; }
interface TopicLite { slug: string; code: string; title: string; tag: Tag;
  source: Source; status: 'todo'|'done'; confidence: number|null; }

interface TopicDetail extends TopicLite {
  deepDive: { concept: string; points: string[]; angle: string };
  note: Note; resources: Resource[]; exercises: Exercise[]; questions: Question[];
}
interface Note { contentMd: string; updatedAt: string;
  attachments: { id: number; url: string; originalName: string; mimeType: string }[]; }
interface Resource { id: number; label: string; url: string; }
interface Exercise { id: number; title: string; repoUrl: string; done: boolean; }
interface Question { id: number; text: string; }

interface Interview { id: number; company: string; role: string;
  stage: 'applied'|'screening'|'onsite'|'offer'|'rejected';
  round: string; scheduledAt: string;
  outcome: 'pending'|'debrief'|'passed'|'offer'|'rejected';
  jdText?: string; weak: string[]; /* topic slugs */ }
interface InterviewQuestion { id: number; text: string; topicSlug?: string; selfRating: number; }
interface PrepPack { interviewId: number; summary: string;
  topics: { topicSlug: string; why: string }[]; generatedAt: string; }

interface StarStory { id: number; title: string;
  situation: string; task: string; action: string; result: string;
  tags: string[]; mappedPrompts: string[]; }

interface MockSession { id: number; type: 'technical'|'system-design'|'behavioral';
  topicScope: string[]; transcript: { role: 'ai'|'user'; text: string }[];
  score: number; feedback: string; createdAt: string; }

interface Job { id: number; company: string; role: string; location: string;
  comp?: string; via: string; date: string; fit: number; reason: string;
  tags: string[]; status: 'new'|'interested'|'dismissed'; }

interface Progress { overallPct: number; streak: number;
  tracks: { name: string; readiness: number }[]; flaggedCount: number; }
interface TodayItem { kind: 'new'|'flagged'|'drill'; topicSlug: string; label: string; note: string; }
interface Usage { monthUsd: number; budgetUsd: number; remainingUsd: number; warning: boolean; }

interface ResumeProfile { skills: {name:string;level:string;years:number}[];
  domains: string[]; seniority: string; totalYears: number;
  experiences: {company:string;role:string;dates:string;highlights:string[]}[];
  gaps: string[]; confirmed: boolean; }
interface OnboardingTargets { targetRole: string; targetLevel: string;
  weeks: number; hoursPerWeek: number; interests: string[];
  monthlyBudgetUsd: number; llmProvider: string; }

```

4. Screens

4.0 Onboarding (new — multi-step wizard)

The first-run flow. Four steps: 1. **Upload resume** — drag-drop PDF → `POST /api/onboarding/resume` → returns draft `ResumeProfile`. 2. **Confirm profile** — editable view of parsed skills/domains/seniority/experiences/**gaps**. User corrects (parsing is lossy) → `PUT /api/onboarding/profile`. 3. **Targets & budget** — form: target role/level, prep weeks, hours/week, interest add-ons (chips, e.g. "Agentic AI"), LLM provider, **monthly budget**. 4. **Review plan** — calls `POST /api/onboarding/plan`; shows generated phases/weeks/topics (DSA auto-leveled, standard tracks, interests). User adds/removes/reorders topics, then `PUT /api/onboarding/plan/commit`. → lands on Home. *Components*: Stepper, FileDrop, EditableProfile, ChipMultiSelect, BudgetField, PlanPreview(editable tree).

4.1 Home dashboard

Greeting + streak + **readiness score** (overall ring) and **per-track readiness** bars. 2×2 pillar cards (Study, Practice, Resources, Interviews) with status. **Today's focus** strip (new + flagged + drill). **Usage meter** (spend vs budget) with amber warning when over. *Calls*: `GET /api/progress`, `GET /api/review/today`, `GET /api/usage`.

4.2 Study plan

Left sidebar: phases with progress bars. Main: selected phase → weeks (expandable) → topics with checkbox, source/tag chips, confidence dot. "Add topic" button (custom). *Calls*: `GET /api/phases`; `PATCH /api/topics/{slug}` (toggle done); `POST /api/topics`.

4.3 Topic detail (the workspace)

Header: title, tag/source, confidence, "Generate guide" (AI) button. Deep-dive panel (concept/points/angle). **Tabs**: - **Notes** — markdown editor (Notion-style), drag-drop images → `POST .../note/attachments`, autosave → `PUT .../note`. - **Resources** — link list (CRUD). - **Exercises** — list with GitHub link (opens repo) + **Monaco viewer** for inline code peek; done toggle. - **Interview questions** — study Q-bank (CRUD). *Calls*: `GET /api/topics/{slug}`, note/resource/exercise/question endpoints, `GET /api/ai/guide/{slug}`.

4.4 Interviews (pipeline + debrief)

Board grouped by stage (applied → screening → onsite → offer/rejected), cards draggable between stages (`PATCH`). "Log interview" — includes a **JD field** (paste if not from a job). **Debrief detail**: questions asked, each tagged to a topic with 1–5 dots; callout "weak topics sent back to plan". **"Generate prep pack"** button → focus topics from JD n weak n gaps. *Calls*: `GET /api/interviews`, `GET /api/interviews/{id}`, `POST/PATCH /api/interviews`, `POST /api/interviews/{id}/questions`, `POST/GET /api/interviews/{id}/prep-pack`.

4.5 Mock interviewer (new)

Pick type (technical / system-design / behavioral) + scope (auto = weak topics). Chat-style timed session with the AI interviewer; ends with **score + feedback**; low areas flag back. *Calls*: `POST /api/mocks/start`, `POST /api/ai/mock/...` (turns), `GET /api/mocks/{id}`.

4.6 Jobs (relevant feed)

Postings parsed from Gmail, **scored 1–10 vs your experience** with a one-line AI reason. Sort by fit/recent; mark interested/dismissed; "log as interview" shortcut (carries JD). *Calls*: `GET /api/jobs`, `PATCH /api/jobs/{id}`.

4.7 STAR stories (new)

Behavioral story bank: cards with Situation/Task/Action/Result, theme tags, mapped prompts. Calls: `CRUD /api/star-stories`.

4.8 Settings (new)

Profile, target role/level, prep duration, hours/week, interests, **LLM provider + model, monthly budget** (with current usage), data **export** (JSON), resume re-upload. Calls: `GET /api/me`, settings update, `GET /api/usage`.

5. Cross-cutting UI behavior

- **Budget warning:** any AI response with `meta.budgetWarning` → show an amber banner "Running on economy mode — monthly budget reached." Non-blocking.
 - **AI affordances:** every "Generate guide / prep pack / mock" button shows a clear loading state and notes it uses AI (cost-aware). Cached results return instantly.
 - **Adaptive cues:** flagged-from-interview items get an amber marker and a "from {company}" note wherever they appear (today's focus, topic list).
 - **Empty/first-run:** onboarding gates the app; until plan committed, redirect to wizard.
 - **Optimistic toggles** for done/confidence; rollback on error.
-

6. Component inventory (shared)

AppShell (nav + header + usage chip) · ReadinessRing · TrackBars · PillarCard · TodayStrip · PhaseSidebar · WeekAccordion · TopicRow · TagChip/SourceChip · DeepDivePanel · MarkdownEditor · MonacoViewer · ResourceList · ExerciseList · QuestionBank · PipelineBoard · InterviewCard · DebriefCard · RatingDots · PrepPackPanel · MockChat · ScoreCard · JobCard · FitBadge · StarStoryCard · Stepper · FileDrop · EditableProfile · ChipMultiSelect · BudgetMeter · Banner.

7. Build order (parallel to backend slices)

1. AppShell + auth + routing; wire to `/api/me`.
2. **Study plan + Topic detail** (against backend slice 2/3) — the core daily surface.
3. Home dashboard (progress, today, usage).
4. Interviews pipeline + debrief (+ JD field, prep pack).
5. Onboarding wizard (once AI parse/plan endpoints exist).
6. Mock interviewer, jobs feed, STAR bank, Settings.

Start with the study + topic screens against the live API as soon as backend slice 2 is up, so both sides validate the contract early.